

Operating Systems

Lab experience - fork() and friends

Mirco Marchetti, Riccardo Lancellotti

SECloud research group
University of Modena and Reggio Emilia

AY 2025/26

Goals of this lab experience:

- Familiarize with the virtual machine that we will use throughout the course
- Create and manage processes in Linux using C programming language
- Understand the concepts of parent and child processes, process hierarchy
- Understand the concepts of orphan and zombie processes
- Use the `fork()`, `wait()`, `getpid()`, `getppid()` and `exec()` system calls

Warm-up exercises

- Replicate the simple examples shown in previous lectures
 - In Moodle, within the section Esercitazioni e dimostrazioni → Processi, you can find the source code of examples showcased in the lectures
- For each example:
 - Take a look at the source and try to understand what it does
 - Compile and run the code
 - Analyze the output and try to understand why it is like that
 - If something is not clear, take a look at the slides, at your notes, at documentation (manpages or online resources)
 - If you still have doubts, ask for help to the teaching staff
- Suggested order: `fork_example.c`, `fork_example_2.c`, `wait_example.c`, `process_tree.c`, `multiple_fork.c`

Example: how to download fork_example.c

- Go to `mydesk.unimore.it`, log in with your credentials and open the virtual machine called **SIST. OP. + RETI**
- Within the virtual machine, open Firefox
Applications → Internet → Firefox Web Browser
- Go to `moodle.unimore.it` and log in with your credentials
- Go to the Moodle course Sistemi Operativi, scroll down to the section Esercitazioni e dimostrazioni, and open the subsection Processi
- Click on the file `fork_example.c` to download it. It should now be saved in `/home/utente/Downloads/fork_example.c`

Example: how to compile and execute fork_example.

- Double click on the Terminal icon on the desktop of the virtual machine to open a terminal, then execute the following commands

change the current directory to Downloads

`cd Downloads`

list the files in the current directory, you should see fork_example.c

`ls`

open the file with the text editor to read the code. When done, exit with [ESC]:q

`view fork_example.c`

compile the code, this will create an executable file called fork_example

`gcc -o fork_example fork_example.c`

list the files in the current directory, you should see fork_example

`ls`

execute fork example

`./fork_example`

Do-It-Yourself exercises

- The following slides describe a task that you are expected to solve
- Try to solve the task by yourself or in a very small group (no more than three students), using the knowledge you acquired in the lectures and in the warm-up exercises
- If you have doubts, ask for help to the teaching staff
- We will not provide a solution **now**! Candidate solutions for each exercise will be provided in a few days, after you had the chance to solve the exercises by yourself
- You can use the text editor of your choice. The virtual machine includes Visual Studio Code, mousepad, vim and nano
- **NOTE:** learning is your responsibility! If you let ChatGPT or the smarter classmate solve the exercises for you, you will not learn anything!

The highest random number

Exercise 1: the highest random number

- Write a C program that creates `n` child processes using `fork()`, `n` being a positive integer provided as a command line argument
- Each child process generates a random integer number between 0 and 255, and prints it on the standard output, together with its PID
- The parent process waits for all child processes to terminate, collects the random numbers generated by the children, and prints the highest one on the standard output together with the pid of the child that generated it

Exercise 1: hints

- If you cannot read an integer from the command line arguments:
 - for now: set `n` to a fixed, hardcoded value (very sad, isn't it?).
 - after this lab: **learn how to program in C! We are not going over the basics of C programming in this course!**
- Call `fork()` in a loop to create `n` child processes
- Use `wait()` in a loop to wait for all child processes to terminate
- Use `exit()` in the child processes to return the generated random number to the parent process
- Use the `WEXITSTATUS()` macro in the parent process to extract the random number returned by each child process
- Use `srand()` to seed the random number generator. Use a different seed for each child process (e.g., its pid)
- Use `rand()` function generate random numbers

Exercise 1: hints

- As a reference, here is a simple C program that generates a random number between 0 and 255, and returns it to the parent process using `exit()`
- We expect you to be able to write a similar program by yourself

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    srand(getpid());
    int r = rand() % 256;
    printf("Random number: %d (PID: %d)\n", r, getpid());
    exit(r);
}
```

Process foster care

Exercise 2: orphan processes

- Write a C program that creates n child processes using `fork()`, n being a positive integer provided as a command line argument
- Each child process:
 - prints its PID and the PID of its parent process on the standard output
 - sleeps for 10 seconds
 - prints its PID and the PID of its parent process on the standard output again
- The parent process:
 - sleeps for 5 seconds
 - terminates without waiting for the child processes to terminate
- Run the program and analyze the output. What do you observe? Why?

Exercise 2: hints

- Who adopted all childrens?
- Try the commands

```
cat /proc/<pid>/cmdline
```

```
ps -xq <pid>
```

where <pid> is the process id of the foster parent

Zombieland

Exercise 3: zombie processes

- Write a C program that creates `n` child processes using `fork()`, `n` being a positive integer provided as a command line argument
- Each child process:
 - prints its PID and the PID of its parent process on the standard output
 - terminates
- The parent process:
 - sleeps for 30 seconds
 - waits for all child processes to terminate
- In a separate terminal, use the `ps a` command to check the status of the processes **while the parent process is sleeping**. Focus on the first (capital) letter of the **STAT** column of the output, which indicates the state of each process. What do you observe? Why?

- Here is the meaning of the first letter of the **STAT** column in the output of the `ps a` command:
 - R: running or runnable (on run queue)
 - S: interruptible sleep (waiting for an event to complete)
 - D: uninterruptible sleep (usually IO)
 - Z: **zombie** (terminated but not reaped by its parent process)
 - T: stopped (on a signal) or traced (during ptrace debugging)
- When a child process terminates, it becomes a zombie until its parent process calls `wait()` to collect its exit status. If the parent process does not call `wait()`, the child process remains a zombie, consuming system resources and potentially leading to resource exhaustion if many child processes are created and left as zombies.

**I've paid for n cores,
I wanna use all n cores!**

Exercise 4: parallel processing

- Write a C program that takes a variable number (at least one) of numbers as command line arguments
- For each number provided as a command line argument, the program creates a child process using `fork()`
- Each process will try to check if the number provided as command line argument is prime or not.
 - If it is prime, the child prints its pid, the number and the message “is prime” on the standard output. It also returns 1 to the parent process using `exit()`
 - If it is not prime, the child process does not print anything and returns 0 to the parent process using `exit()`
- The parent process waits for all child processes to terminate, collects the return values of the child processes, and prints the total number of prime numbers

Exercise 4: hints

- Use `long` instead of `int` to store the numbers provided as command line arguments, since they can be very large
- Use the `atoi()` function to convert the command line arguments from strings to long integers
- Below you can find a very simple (and inefficient...) implementation of a primality test. We expect you to be able to write a similar program by yourself.

```
static int is_prime(long x) {  
    if (x < 2) return 0;  
    if (x == 2) return 1;  
    if (x % 2 == 0) return 0;  
    for (long d = 3; d * d <= x; d += 2) {  
        if (x % d == 0) return 0;  
    }  
    return 1;  
}
```

Exercise 4: hints

- To make things more interesting we can use some not-so-trivial prime numbers (possibly multiple times). A few examples:

99999999999999937

611091996016611019

1234567894987654321

1241111124211111421

- Be careful not to exceed the maximum value of a long integer, which is 9223372036854775807 on a 64-bit system. If you try to convert a number larger than this using `atoi()`, you will get undefined behavior (`strtol()` has a much better behavior, you should use it...).
- In a separate terminal, use the `top` command to check the CPU usage of the processes while they are running. What do you observe? Why?

Shell we begin?

Exercise 5: build a simple shell

- Write a C program that implements a simple shell. The shell should repeatedly prompt the user for a command, read the command from the standard input, and execute it using `fork()` and `exec()`
- No frills: no history, no environment variables, no pipes, no redirections, no built-in commands, etc. Just a simple shell that can execute simple commands like `ls`, `cat`, `echo`, etc.
- No path management: the user should provide the full path of the command to execute, for example `/bin/ls` instead of just `ls`
- The shell should terminate when the user types `exit`

Exercise 5: hints

- Use a main loop to repeatedly prompt the user for a command and read it from the standard input
- Use the `strtok()` function to parse the command and its arguments from the input string
- Use `fork()` to create a child process that will execute the command, and a syscall of the `exec()` family to replace the child process with the command to execute
- Use `wait()` in the parent process to wait for the child process to terminate before prompting the user for the next command

Exercise 5: hints

- Too hard? Start simple!
- First create a process that creates a child that uses `execv()` to execute a hardcoded command without parameters (e.g. `/bin/ls`)
- Then make it execute a hardcoded command with a hardcoded set of parameters (e.g. `ls -l`)
- Then mess around with `strtok()` to understand how to tokenize a string
- Then put it all together!